NGUYEN DEVELOPMENT · PHPDBADMIN

Evolutionary Development — phpdbadmin

Timeboxes und Architekturentscheide ADR-001 bis ADR-011

Phase	04 Evolutionary Development
Träger	Nguyen Development (Einzelfirma), Hoang Nguyen · Lizenz MIT
Vorgehen	DSDM/AgilePM — Dokumentation in dokumentationen/, ein PDF je Phase
Stand	28.10.2026, 21:27 · baut auf Commit 5af795c + Änderungen dieses Standes auf
Quelle	dokumentationen/ im Repo — dieses PDF wird bei jeder Änderung neu gebaut

Dieses Dossier enthält

1. 04 — Evolutionary Development
2. Anhang A — ADR-001 — Dialekt-Schicht: ein Vertrag, vier Umsetzungen
3. Anhang B — ADR-002 — Zielverbindungen je Anfrage, Grenzen und Abbruch je Treiber
4. Anhang C — ADR-003 — Vorschau aus Blueprint::toSql(), Anwenden je Treiber
5. Anhang D — ADR-004 — Schreibschutz in Schichten und eine Anweisung je Aufruf
6. Anhang E — ADR-005 — SQL-Editor: CodeMirror 6
7. Anhang F — ADR-006 — Raster: TanStack Virtual und Blättern über den Schlüssel
8. Anhang G — ADR-007 — Export und Import in Paketen im Hintergrund
9. Anhang H — ADR-008 — Anmeldung ohne Passwort
10. Anhang I — ADR-009 — App-Datenbank: SQLite als Vorgabe
11. Anhang J — ADR-010 — Lizenz MIT
12. Anhang K — ADR-011 — PHP 8.5 als Mindestversion

Inhalt

04 — Evolutionary Development

Geplante erste Timebox
Architekturentscheide

Anhang A — ADR-001 — Dialekt-Schicht: ein Vertrag, vier Umsetzungen

Kontext
Entscheid
Optionen
Konsequenzen

Anhang B — ADR-002 — Zielverbindungen je Anfrage, Grenzen und Abbruch je Treiber

Kontext
Entscheid
Optionen
Konsequenzen

Anhang C — ADR-003 — Vorschau aus Blueprint::toSql(), Anwenden je Treiber

Kontext
Entscheid
Optionen
Konsequenzen

Anhang D — ADR-004 — Schreibschutz in Schichten und eine Anweisung je Aufruf

Kontext
Entscheid
Optionen
Konsequenzen

Anhang E — ADR-005 — SQL-Editor: CodeMirror 6

Kontext
Entscheid
Optionen
Konsequenzen

Anhang F — ADR-006 — Raster: TanStack Virtual und Blättern über den Schlüssel

Kontext
Entscheid
Optionen
Konsequenzen

Anhang G — ADR-007 — Export und Import in Paketen im Hintergrund

Kontext
Entscheid
Optionen
Konsequenzen

Anhang H — ADR-008 — Anmeldung ohne Passwort

Kontext
Entscheid
Optionen
Konsequenzen

Anhang I — ADR-009 — App-Datenbank: SQLite als Vorgabe

Kontext
Entscheid
Optionen
Konsequenzen

Anhang J — ADR-010 — Lizenz MIT

Kontext
Entscheid
Optionen
Konsequenzen

Anhang K — ADR-011 — PHP 8.5 als Mindestversion

Kontext
Entscheid
Optionen
Konsequenzen

04 — Evolutionary Development

> **PDF der Phase:** evolutionary-development.pdf — bei jeder Änderung neu gebaut (`python3 dokumentationen/werkzeuge/pdf-bauen.py --nur 04`).

📋 **Platzhalter** — Timebox-Protokolle (`timebox-nn-titel.md`) und Architekturentscheide (`adr/ADR-nnn-titel.md`: Kontext → Optionen → Entscheid → Konsequenzen).

Geplante erste Timebox

TB	Inhalt	Anforderungen
00 Fundament — Scheiben im Timebox-Plan § 2	Starter-Kit bereinigen: Passwort-Anmeldung, Registrierung, Passwort-Reset und -Bestätigung, Zwei-Faktor, E-Mail-Bestätigung, **Dashboard** und **Profil** entfernen; Anmeldung mit Code per Mail und Passkey; Einladung und Erst-Setup per Konsole; App-Datenbank; Verbindungen mit Test und Verschlüsselung; Dialekt-Schicht mit Vertragstest; MySQL in Herd; Tests auf vier Datenbanken; Workflow sparsam	A-01 bis A-06, A-09, V-01 bis V-03, V-05, B-01 bis B-03, B-06, B-08

Architekturentscheide

Ausformuliert in der Foundation am 28.10.2026 (Kontext → Optionen → Entscheid → Konsequenzen), Grundlage Feasibility und Solution-Architektur:

ADR	Entscheid	Status
ADR-001	Dialekt-Schicht: ein Vertrag, vier Umsetzungen	angenommen
ADR-002	Zielverbindungen je Anfrage, Grenzen und Abbruch je Treiber	angenommen
ADR-003	Vorschau aus `toSql()`, Anwenden je Treiber	angenommen
ADR-004	Schreibschutz in Schichten, eine Anweisung je Aufruf	angenommen
ADR-005	SQL-Editor CodeMirror 6	angenommen
ADR-006	Raster TanStack Virtual, Blättern über den Schlüssel	angenommen
ADR-007	Export und Import in Paketen im Hintergrund	angenommen
ADR-008	Anmeldung ohne Passwort	angenommen
ADR-009	App-Datenbank SQLite als Vorgabe	angenommen
ADR-010	Lizenz MIT	angenommen
ADR-011	PHP 8.5 als Mindestversion	angenommen (Hoang 28.10.2026)

Übernahme aus der Feasibility in TB 00: Probe-Seiten `routes/probe.php`, `resources/js/pages/probe/*` übernehmen oder entfernen; die Proben in `tests/Feature/Feasibility` werden zu Vertragstests der Dialekt-Schicht.

Anhang A — ADR-001 — Dialekt-Schicht: ein Vertrag, vier Umsetzungen

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-1, F-3)
Betrifft	Katalog, Struktur, SQL-Editor, Server-Verwaltung

Kontext

phpdbadmin soll vier Datenbanken mit derselben Bedienung abdecken (P-01). Laravel 13 liefert Verbindungen, Query Builder, Schema-Grammatiken und Introspektion für alle vier; die Feasibility zeigt, wo es nicht reicht: Check-Constraints, Trigger, Routinen, eigene Typen (`getTypes()` nur PostgreSQL), Prozesse, Rechte, Zeitlimits, Kontorechte. Ausserdem liefert MariaDB ohne Schemaangabe die Tabellen aller Datenbanken des Servers (3'603 statt 1'000).

Optionen

Option	Vorteile	Nachteile
A — eigene Abstraktion für alles	volle Kontrolle	baut nach, was Laravel schon kann und testet; viermal so viel Code
B — Laravel zuerst, Dialekt nur für Unterschiede	wenig eigener Code, Laravel-Updates kommen mit	Grenze zwischen Laravel und Dialekt muss klar sein
C — Paket eines Dritten (z. B. Adminer-Treiber)	viel fertig	fremde Struktur, Lizenzfrage, Bedienkonzept passt nicht

Entscheid

Option B. Ein Vertrag `App\Dialects\Dialect` mit vier Umsetzungen (`MySqlDialect`, `MariaDbDialect` als Ableitung, `PostgresDialect`, `SqliteDialect`). Was Laravel kann, ruft der Dienst direkt; der Dialekt liefert nur, was sich unterscheidet. Der Katalog übergibt **immer** das Schema.

Konsequenzen

- Jede Dialekt-Methode hat einen Vertragstest, der gegen alle vier Herd-Dienste läuft; die Proben aus `tests/Feature/Feasibility` werden dazu umgebaut.
- Neue Datenbanken (SQL Server, R-14) bekommen eine fünfte Umsetzung, die Oberfläche bleibt.
- Typnamen erscheinen im Original und mit einem gemeinsamen Oberbegriff.

Siehe Solution-Architektur · Feasibility

Anhang B — ADR-002 — Zielverbindungen je Anfrage, Grenzen und Abbruch je Treiber

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-2, F-8)
Betrifft	ConnectionFactory, QueryRunner

Kontext

Zugangsdaten liegen verschlüsselt in der App-Datenbank. Jede Anfrage braucht eine Verbindung mit Zeitzone UTC, Zeitlimit und gegebenenfalls Schreibschutz. Die Feasibility zeigt grosse Unterschiede: MySQL begrenzt nur SELECT und bricht still ab (Ergebnis `1`), MariaDB und PostgreSQL melden 70100 bzw. 57014, SQLite kennt kein Zeitlimit; libpq hält ganze Ergebnisse ausserhalb des PHP-Speichers (+105 MB bei einer Million Zeilen).

Optionen

Option	Vorteile	Nachteile
A — Verbindungen in `config/database.php` eintragen	einfach	Neustart je Verbindung, Geheimnisse in Dateien
B — `DB::build()` je Anfrage aus der App-Datenbank	keine Dateien, Name je Verbindung und Konto, `DB::purge` danach	Verbindungsaufbau je Anfrage (lokal ~1 ms)
C — dauerhafte Verbindungen (Pool)	schneller	PHP-FPM hat keinen gemeinsamen Pool; Zustand zwischen Konten

Entscheid

Option B. `ConnectionFactory` baut je Anfrage mit `timezone`, Zeitlimit (`max_execution_time`, `max_statement_time`, `statement_timeout`) und Optionen. Zeilengrenze je Treiber: MySQL/MariaDB ungepuffert + `KILL QUERY`, PostgreSQL Server-Cursor (`DECLARE … CURSOR`, `FETCH`), SQLite streamt. SQLite-Abfragen aus dem Editor laufen im Arbeitsprozess mit Zeitgrenze. Abbrechen ist eine zweite Anfrage; ein Merker im Cache entscheidet über «abgebrochen», nicht der Fehlercode.

Konsequenzen

- Verbindungsfehler werden je Treiber zugeordnet (MySQL nach Code, PostgreSQL nach Meldungstext).
- Der Arbeitsprozess für SQLite braucht die Warteschlange oder einen Kindprozess; ohne beides gilt die PHP-Laufzeitgrenze.
- Erlaubte Hosts werden vor dem Verbinden geprüft (ADR-004, V-05).

Siehe Solution-Architektur · Feasibility

Anhang C — ADR-003 — Vorschau aus Blueprint::toSql(), Anwenden je Treiber

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-5)
Betrifft	SchemaEditor, ChangeSet

Kontext

Nichts soll ungesehen passieren (P-02). Die Feasibility belegt: `toSql()` ist auf allen vier genau das, was läuft; `pretend()` liefert für DML eine gute Vorschau, für SQLite-DDL aber eine kaputte. Naiv in einer Transaktion angewendet, wirft Laravel auf MySQL/MariaDB «There is no active transaction» (DDL beendet Transaktionen implizit), und **SQLite löscht beim Umbau einer Elterntabelle alle abhängigen Zeilen** (0 von 2), weil `PRAGMA foreign_keys` in einer Transaktion nicht wirkt.

Optionen

Option	Vorteile	Nachteile
A — alles in einer Transaktion	einfach, scheinbar sicher	auf drei von vier Treibern falsch, auf SQLite Datenverlust
B — Verfahren je Treiber	auf allen vier belegt korrekt (2 von 2 Kindern bleiben)	drei Wege, die Vorschau muss sie erklären
C — Strukturänderungen nur über eigenes SQL	keine Abhängigkeit von Laravel	baut die Grammatiken nach, die SQLite-Umbau schon können

Entscheid

Option B. Vorschau von Strukturänderungen aus `Blueprint::toSql()`, von Datenänderungen aus `pretend()`. Anwenden: MySQL/MariaDB Anweisung für Anweisung ohne Transaktion (Vorschau: «nicht rückrollbar»); PostgreSQL in einer Transaktion; SQLite nach sqlite.org: Fremdschlüssel ausserhalb aus, Umbau in der Transaktion, `foreign_key_check`, Commit, ein. Datenänderungen auf allen vier in einer Transaktion, vorher Sicherung für Rückgängig.

Konsequenzen

- Tests für Strukturänderungen immer mit Elterntabelle und abhängigen Zeilen.
- Bei MySQL/MariaDB meldet die Oberfläche nach einem Fehler, welche Schritte schon gelaufen sind.
- Lektion «SQLite-Tabellenumbau nie in eigener Transaktion» im Vault.

Siehe Solution-Architektur · Feasibility

Anhang D — ADR-004 — Schreibschutz in Schichten und eine Anweisung je Aufruf

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-7)
Betrifft	ReadOnlyGuard, QueryRunner, SQLite-Verbindungen

Kontext

Produktion ist standardmässig schreibgeschützt (V-04), und der SQL-Editor nimmt freies SQL. Ein Schreibschutz über eine Sitzungseinstellung allein lässt sich im Editor abschalten (`SET … READ WRITE`, `PRAGMA query_only = OFF`). Die Feasibility hat einen Wächter mit 5–12 Ausbruchsversuchen je Treiber geprüft: Bestand unverändert.

Optionen

Option	Vorteile	Nachteile
A — nur Lesekonto verlangen	das einzige echte Schloss	viele haben nur ein Konto; lokal unpraktisch
B — Schichten: Lesekonto empfohlen, Nur-lesen-Transaktion je Anweisung, vorbereitet, SQLite nur lesend geöffnet + Authorizer	belegt auf allen vier	jede Anweisung einzeln; Zerleger nötig
C — SQL vor dem Ausführen klassifizieren und Schreibendes ablehnen	ohne Transaktion	Parser-Rennen gegen jede Syntax; Funktionen mit Nebenwirkung

Entscheid

Option B. MySQL/MariaDB: `SET SESSION TRANSACTION READ ONLY` + Transaktion, immer zurückrollen; PostgreSQL: Transaktion + `SET TRANSACTION READ ONLY`, zurückrollen; SQLite: `Pdo\Sqlite::OPEN_READONLY` und `setAuthorizer()` (`ATTACH` nur im erlaubten Ordner). Anweisungen werden auf dem Server zerlegt und einzeln vorbereitet; `unprepared()` ist für Eingaben verboten. Konten mit Superuser- oder Dateirechten werden sichtbar gewarnt (V-10).

Konsequenzen

- Der Zerleger je Dialekt ist Voraussetzung (TB 03, Probe zu Beginn).
- Die Ausbruchsversuche aus `SecurityProbeTest` werden Abnahmetest von V-04.
- PHP 8.5 wird Mindestversion (ADR-011).

Siehe Solution-Architektur · Feasibility

Anhang E — ADR-005 — SQL-Editor: CodeMirror 6

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-4, Freigabe der Pakete durch Hoang)
Betrifft	Oberfläche SQL-Reiter

Kontext

Der Editor braucht Hervorhebung und Vervollständigung je Dialekt aus dem Schema (Q-01). Probeweise installiert: `codemirror` 6.0.2, `@codemirror/lang-sql` 6.10.0 (MIT, 17 Pakete). Belegt: Tabellen aus 1'002 Tabellen in höchstens 2 ms, Aliasse aufgelöst, dialekteigene Schlüsselwörter (`ILIKE` nur PostgreSQL); Chunk 425 KB (138 KB gzip).

Optionen

Option	Vorteile	Nachteile
A — CodeMirror 6	modular, MIT, Dialekte für alle vier, klein genug	eigene Vue-Hülle
B — Monaco	VS-Code-Gefühl	mehrere MB, SQL-Dialekte schwächer, Web-Worker
C — Eigenbau mit Textarea	keine Abhängigkeit	keine Vervollständigung, viel Arbeit

Entscheid

Option A, als eigener Chunk nur auf dem SQL-Reiter geladen.

Konsequenzen

- Sichtprüfung im Browser durch Hoang steht aus (Probe-Seite `/probe/editor`).
- Bei Updates der Pakete Lizenzliste neu erzeugen (ADR-010).

Siehe Solution-Architektur · Feasibility

Anhang F — ADR-006 — Raster: TanStack Virtual und Blättern über den Schlüssel

Feld	Wert
Status	Angenommen — bestätigt von Hoang am 28.10.2026 (Endlos scrollen statt Seiten zu 50)
Datum	28.10.2026 (Foundation; F-3)
Betrifft	Datenraster

Kontext

Tabellen mit Millionen Zeilen müssen sofort erscheinen (N-01). Gemessen: erste Seite über den Schlüssel 0,1–0,6 ms in der Datenbank, `OFFSET 900000` 18–232 ms; im Browser 281 ms für die erste Seite, davon fast alles Laravel und Herd; im DOM bleiben rund 25 Zeilen.

Optionen

Option	Vorteile	Nachteile
A — eigenes Raster mit `@tanstack/vue-virtual` (MIT, 34 KB Chunk)	volle Kontrolle über Bearbeiten, Zustände, Gestaltung	Raster selbst bauen
B — fertiges Datenraster (AG Grid Community u. a.)	viel fertig	schwer, eigene Bedienlogik, Gestaltung schwer anzugleichen
C — ohne Virtualisierung, Seiten zu 100	einfach	kein flüssiges Scrollen, Seitenwechsel

Entscheid

Option A. Seiten zu 200 Zeilen über den Schlüssel, die nächste vorgeladen; `OFFSET` nur ohne Primärschlüssel; erste Spalte fixiert, waagrecht scrollen nur hier.

Konsequenzen

- Geänderte, neue und zu löschende Zeilen sind Zustand im Browser bis «Anwenden» (D-04).
- Kartenliste unter 768 px (G-07).

Siehe Solution-Architektur · Feasibility

Anhang G — ADR-007 — Export und Import in Paketen im Hintergrund

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-8, F-3)
Betrifft	Transfer

Kontext

Gemessen bei einer Million Zeilen: `lazyById(5000)` 3–3,3 MB auf allen vier; `cursor()` gepuffert 62 MB (MySQL/MariaDB), PostgreSQL +105 MB im Prozess. Import von 100'000 Zeilen in Paketen zu 1'000: 143–502 ms. Nach dem Füllen fehlt MySQL/MariaDB die Statistik — Sortieren über den Index 300 statt 2 ms.

Optionen

Option	Vorteile	Nachteile
A — Pakete über den Schlüssel, im Hintergrund, Datei mit Frist	gleich auf allen vier, wenig Speicher	Warteschlange nötig (ohne Worker synchron mit Grenze)
B — Server-Werkzeuge (`mysqldump`, `pg_dump`)	schnell, vollständig	nicht überall installiert, Versionen müssen passen — nur für Sicherung (E-03)

Entscheid

Option A für Export und Import; Probelauf beim Import in einer zurückgerollten Transaktion; nach eigenen Importen `ANALYZE` (E-06). Server-Werkzeuge nur für Sicherung und Wiederherstellung (E-03, v1.1).

Konsequenzen

- Exportdateien unter `storage/app/private/transfers`, Frist 24 Stunden.
- Tabellen ohne Primärschlüssel exportieren über `cursor()` ungepuffert bzw. Server-Cursor.

Siehe Solution-Architektur · Feasibility

Anhang H — ADR-008 — Anmeldung ohne Passwort

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Vorgabe Hoang; F-6)
Betrifft	Anmeldung, Konten

Kontext

Vorgabe: kein Passwort, nur Passkey und Code per Mail; kein Dashboard, kein Profil. Fortify 1.41 registriert `POST /login` und die Passwortbestätigung unabhängig von den Features; das Feature `passkeys` steht im Starter-Kit mit `confirmPassword: true`. Basalto hat dasselbe mit denselben Versionen gebaut (Code-Login, Abweisung, Software-Authenticator für Tests).

Optionen

Option	Vorteile	Nachteile
A — Fortify nur für Passkeys, eigener Code-Login, Passwort-Endpunkte abweisen	Muster erprobt, Passkeys von Laravel	Fortify-Updates können Routen verschieben (Lektion)
B — Fortify ganz entfernen, Passkeys selbst	keine Überraschungen aus Fortify	WebAuthn selbst prüfen — mehr Risiko

Entscheid

Option A. `Fortify::authenticateUsing()` liefert `null`; Passwortbestätigung durch Bestätigung per Passkey bzw. Code ersetzt; Passwort-Spalten und -Tabellen entfernt; erstes Konto per `phpdbadmin:install`, weitere per Einladung.

Konsequenzen

- Test ruft jede alte Adresse des Starter-Kits auf (404/405).
- Vor jedem `composer update` Routenliste sichern und vergleichen.
- Parameter nie `{passkey}` nennen (globale Bindung von `laravel/passkeys`).

Siehe Solution-Architektur · Feasibility

Anhang I — ADR-009 — App-Datenbank: SQLite als Vorgabe

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; R-31)
Betrifft	Installation, Betrieb

Kontext

Die App-Datenbank hält Konten, Verbindungen, Verlauf und Protokoll — wenig Daten, wenige gleichzeitige Schreiber. Eine Installation neben fremden Datenbanken soll ohne zusätzlichen Server laufen.

Optionen

Option	Vorteile	Nachteile
A — SQLite als Vorgabe, die anderen drei möglich	keine Einrichtung, Sicherung = eine Datei	Schreibsperren bei vielen Parallelzugriffen (`busy_timeout`, WAL)
B — MariaDB/PostgreSQL vorgeschrieben	robuster bei vielen Konten	zusätzlicher Server nur für phpdbadmin

Entscheid

Option A, mit WAL und `busy_timeout`; die Testsuite läuft mit jeder der vier als App-Datenbank (B-06).

Konsequenzen

- Regeln für Spaltentypen gelten trotzdem für alle vier (Datenmodell § 1).
- Die App-Datenbank erscheint nie als Zielverbindung (R-17).

Siehe Solution-Architektur · Feasibility

Anhang J — ADR-010 — Lizenz MIT

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Entscheid Hoang, R-40; F-9)
Betrifft	Repository, Abhängigkeiten

Kontext

Hoang hat MIT gewählt (Markt überwiegend kostenlos, Schaufenster für Nguyen Development). Geprüft: Composer 101 Pakete MIT/BSD/Apache, `nette/*` wahlweise BSD-3; npm-Laufzeit MIT/ISC/BSD/Apache; neue Pakete MIT.

Optionen

Option	Vorteile	Nachteile
A — MIT	einfach, verbreitet	kein Schutz gegen geschlossene Ableitungen
B — GPL-3	Ableitungen bleiben offen	schreckt Einsatz in Firmen ab

Entscheid

Option A. `LICENSE` mit MIT vor dem ersten öffentlichen Push; Code aus GPL-Projekten (phpMyAdmin, phpPgAdmin, Adminer) wird nicht übernommen.

Konsequenzen

- Lizenzliste vor jedem Release erneut erzeugen.
- Hinweis «nicht verbunden mit phpMyAdmin» in README und Website (F-10).

Siehe Solution-Architektur · Feasibility

Anhang K — ADR-011 — PHP 8.5 als Mindestversion

Feld	Wert
Status	Angenommen (Hoang, 28.10.2026)
Datum	28.10.2026 (Foundation)
Betrifft	Installation, Hosting, `composer.json`

Kontext

Der Schutz von SQLite gegen `ATTACH` fremder Dateien braucht `Pdo\Sqlite::setAuthorizer()`. Geprüft in Herd: PHP 8.4.27 hat `Pdo\Sqlite`, aber keinen Authorizer; PHP 8.5.12 hat ihn. Laravel 13 verlangt nur PHP 8.3, `composer.json` steht auf `^8.3`.

Optionen

Option	Vorteile	Nachteile
A — PHP 8.5 vorschreiben	Schutz vollständig, ein Weg	Hosting mit PHP 8.4 fällt weg
B — PHP 8.4 erlauben, SQLite-`ATTACH` dort per Textprüfung sperren	breiter einsetzbar	Textprüfung ist schwächer; zwei Wege testen
C — PHP 8.4 erlauben, SQLite-Verbindungen dort nur lesend und ohne Editor	einfach	halbe Funktion

Entscheid

Option A (entschieden von Hoang am 28.10.2026). `composer.json` auf `^8.5` in TB 00; Herd läuft bereits mit 8.5.

Konsequenzen

- Die Installationsanleitung nennt PHP 8.5 und `pdo_sqlite`, `pdo_mysql`, `pdo_pgsql`.

Siehe Solution-Architektur · Feasibility